LEAP: Hardware-Aware Learned Eviction for Line-Rate Persistent Flow Measurement on FPGAs and Programmable Switches

Technical Report (Supplementary Material)

Anonymous Authors
Supplement to a paper under double-blind review

This report explains the experiments behind the main results of the paper "LEAP: Hardware-Aware Learned Eviction for Line-Rate Persistent Flow Measurement on FPGAs and Programmable Switches." Section S1 bounds the error of LEAP and proves the hardware shortcuts correct, Section S2 shows how the policy is trained and what it learns, Sections S3 and S4 show that the FPGA and Tofino implementations behave like the simulator, and Section S5 adds the results that the paper only summarizes. Unless stated otherwise, the setup is that of Section V of the paper: MAWI 2026 traces, $M = 1500$ windows, $\alpha = 0.4$, $d = 2$ rows, and five hash seeds.

S1. Analysis

This section shows that the error of LEAP on a persistent flow is one-sided, bounds its two parts and the probability that the flow is evicted, explains why presence bits help, evaluates the exact chains of the deployed table, and proves the FPGA shortcuts correct.

A. Model

Fix a persistent flow f. The policy is a function $\pi_h(v)$ of the presence bits $h \in \{00, 01, 10, 11\}$ and the count v in the current time bin; a count-only policy has $\pi_h(v) = \pi(v)$ for all h. Theorems S1–S3 fix the time bin; Section S1-G removes this restriction. We assume:

(A1) f appears in each window independently with probability λ, its *presence rate*; f is persistent if $\lambda \geq \alpha$.

(A2) In each window, independently, with probability ρ, exactly one packet of another flow finds the bucket of f to be its minimum-count candidate, which gives one decay attempt; in a window in which f appears, the attempt precedes the first packet of f with probability θ.

(A3) No other flow has the fingerprint of f.

(A4) The presence bits of the bucket of f, which record windows $t-2$ and $t-1$, follow their distribution under (A1), $w(\lambda) = \big((1-\lambda)^2, \lambda(1-\lambda), (1-\lambda)\lambda, \lambda^2\big)$ for $H = 00, 01, 10, 11$, independently of the count.

(A4) holds for the bits alone from the third window after insertion on; it neglects their correlation with the count, and Section S1-G drops it. Let $\bar{\pi}(v) = \sum_h w_h(\lambda)\, \pi_h(v)$. The probability that, in a given window, f is absent and its bucket loses one unit at count v is

$$q_v = \rho\,(1-\lambda)\,\bar{\pi}(v), \tag{S1}$$

and $q_v^{\mathrm{cnt}} = \rho(1-\lambda)\pi(v)$ for a count-only policy. Both the Pontus rule $\pi(v) = 1/(v+1)$ and the learned policy are non-increasing in v for every h. With c attempts per absent window, $\pi_h(v)$ becomes $1-(1-\pi_h(v))^c$, as a failed draw leaves $R = 0$ and allows another.

B. One-sided error

Lemma S1 (one-sided error)**.** *Under (A3), a flow f that owns a bucket at the end of window t satisfies $\hat{P}(f) \leq P(f)$, for every policy.*

Proof. When f is inserted in window t_0, its count is 1 and f has appeared in t_0. Afterwards, the count changes only through arrivals of f and through decays. An arrival adds to the count only if $F = 0$, i.e., once per window, and adds 2 only if $R = 1$, i.e., only if a decay already removed one unit in the same window; a decay requires $F = R = 0$ and sets R, so at most one happens per window. The net change in a window is therefore $+1$ if f appears and 0 or -1 otherwise, so the count never exceeds the number of windows in which f appeared since t_0. □

The proof gives more: after insertion, the count of f is a lazy random walk that steps up in each window in which f appears and down only in some windows in which it is absent. A decay before the first packet of f in a window in which f appears is undone by the $+2$, except at count 1, where it evicts f. If f is inserted and never evicted, its error splits into two parts,

$$P(f) - \hat{P}(f) = \Delta_1 + \Delta_2, \tag{S2}$$

where Δ_1 is the number of windows in which f appeared before it obtained its bucket (late insertion), and Δ_2 is the number of decays of its bucket in windows in which f is absent (decay loss). If f is never inserted, $P(f) - \hat{P}(f) \leq \Delta_1$.

C. Late insertion

Theorem S1 (entry delay)**.** *Let the candidate bucket of a new flow be owned by a flow of count V that stops appearing, and let the new flow collide with it, as the minimum-count bucket,*

in each window independently with probability ρ and at most once per window. Under a count-only policy π, the expected number of windows until the new flow replaces the owner is

$$\mathbb{E}[D] = \sum_{v=1}^{V} \frac{1}{\rho\,\pi(v)}, \tag{S3}$$

which is $V(V+3)/(2\rho) = \Theta(V^2/\rho)$ for the Pontus rule. Under a presence-aware policy, $\mathbb{E}[D] \le 2 + \sum_{v\le V} 1/(\rho\,\pi_{00}(v))$.

Proof. The absent owner keeps $F = 0$, and R is cleared at every window boundary, so each window with a collision gives exactly one decay attempt, which succeeds with probability $\pi(v)$ at count v and moves the owner to count $v-1$. The number of windows spent at count v is therefore geometric with mean $1/(\rho\,\pi(v))$, and the claim follows by linearity of expectation, with $\sum_{v\le V}(v+1) = V(V+3)/2$. With presence bits, the owner's bits become $H = 00$ two windows after its last appearance; decays in these two windows can only lower the count; later attempts succeed with probability $\pi_{00}(v)$. □

If the new flow is f itself, sending one packet per window as the beacon of Fig. 1 of the paper does, and the owner is already stale ($H = 00$), then every window in which f appears is an attempt and all but the last attempt are missed windows:

$$\mathbb{E}[\Delta_1] = \sum_{v=1}^{V} \frac{1}{\pi_{00}(v)} - 1, \tag{S4}$$

independently of λ; this is $V(V+3)/2 - 1$ for the Pontus rule. A flow that sends c packets per window gets up to c attempts per window, which divides the delay by at most c, so it stays quadratic in V.

D. Decay loss and eviction

Theorem S2 (decay loss and eviction)**.** *Let f own a bucket with count 1, and let q_v in* (S1) *be non-increasing for $v \ge 2$ with $q_2 < \lambda$. Under (A1)–(A4):*

(a) *the expected number of decays of the bucket of f at counts $v \ge 2$ satisfies*

$$\mathbb{E}[\Delta_2'] \le \sum_{v=2}^{M} \frac{q_v}{\lambda - q_v}, \tag{S5}$$

and $\Delta_2 = \Delta_2'$ if f is never evicted;

(b) *the probability that f is ever evicted is*

$$\Pr[\text{evict}] = \frac{\sum_{k\ge1}\gamma_k}{\sum_{k\ge0}\gamma_k}, \qquad \gamma_k = \frac{\tilde{q}_1}{\tilde{\lambda}_1}\prod_{v=2}^{k}\frac{q_v}{\lambda}, \tag{S6}$$

where $\gamma_0 = 1$, $\tilde{q}_1 = q_1 + \rho\theta\lambda\bar{\pi}(1)$, and $\tilde{\lambda}_1 = \lambda(1 - \rho\theta\bar{\pi}(1))$.

For the Pontus rule, $\mathbb{E}[\Delta_2'] \le \rho(1-\lambda)\ln(M+1)/(\lambda - \rho(1-\lambda)/3) = O(\log M)$.

Proof. By Lemma S1 and the remark after it, the count C_t of the bucket is a birth–death chain on $\{0, 1, 2, \dots\}$, absorbed at 0 (eviction). At $v \ge 2$, it steps up with probability λ (a decay before f's packet is undone) and down with probability q_v. At $v = 1$, every decay evicts f: it steps down with probability $\tilde{q}_1$, which adds the decays before f's packet in present windows, and up with probability $\tilde{\lambda}_1$.

(a) Let $v \ge 2$ and N_v be the number of windows the chain spends at v, and let h_{v+1} be the probability of ever reaching v from $v+1$. Because $q_u \le q_{v+1} < \lambda$ for all $u > v$, a coupling with the homogeneous walk with down-probability q_{v+1} and gambler's ruin give $h_{v+1} \le q_{v+1}/\lambda$. In each window at v, the chain steps up with probability λ and then never returns with probability at least $1 - h_{v+1}$, so $\mathbb{E}[N_v] \le 1/(\lambda(1-h_{v+1})) \le 1/(\lambda - q_{v+1}) \le 1/(\lambda - q_v)$, and the expected number of down-steps from v is at most $q_v\,\mathbb{E}[N_v] \le q_v/(\lambda - q_v)$. Summing over the levels reachable in M windows gives (S5). If f is never evicted, no decay happened at count 1, so $\Delta_2 = \Delta_2'$.

(b) This is the absorption probability of a birth–death chain started at 1 with the transition probabilities above (holding does not change it); over M windows, it is smaller.

For the Pontus rule, $q_v = \rho(1-\lambda)/(v+1) \le q_2$ for $v \ge 2$ and $\sum_{2\le v\le M} 1/(v+1) \le \ln(M+1)$, so (S5) is at most $\rho(1-\lambda)\ln(M+1)/(\lambda - q_2)$. □

For a policy that changes with the time bin, Theorems S1 and S2 still hold with $\pi_{00}(v)$ replaced by its minimum over the bins traversed and q_v and $\bar{\pi}(1)$ by their maxima: the proofs only use per-window bounds, and the eviction probability follows from a monotone coupling of the two chains.

E. Error bound

Theorem S3 ((ε, δ) underestimation bound)**.** *Under (A1)–(A4), let a persistent flow f send one packet in each window in which it appears, let its candidate bucket be owned by a stale owner ($H = 00$) of count V when f first appears, let f be the only flow that collides with that owner, and let the other candidate bucket of f never be its minimum-count bucket. Then for every $\varepsilon > 0$,*

$$\Pr\big[P(f) - \hat{P}(f) \ge \varepsilon M\big] \le \delta, \quad \textit{where}$$

$$\delta = \Pr[\text{evict}] + \frac{1}{\varepsilon M}\Big(\sum_{v=1}^{V}\frac{1}{\pi_{00}(v)} - 1 + \sum_{v=2}^{M}\frac{q_v}{\lambda - q_v}\Big). \tag{S7}$$

For the Pontus rule, $\delta = \Pr[\text{evict}] + \big(V(V+3)/2 + O(\log M)\big)/(\varepsilon M)$.

Proof. If f is never evicted, then $P(f) - \hat{P}(f) \le \Delta_1 + \Delta_2'$ by (S2) and Theorem S2(a), and also if f is never inserted. Hence $\Pr[P - \hat{P} \ge \varepsilon M] \le \Pr[\text{evict}] + \Pr[\Delta_1 + \Delta_2' \ge \varepsilon M]$, and Markov's inequality with (S4) and (S5) gives (S7). □

The theorem bounds underestimation only. Under (A3), f can be overestimated only if it owns no bucket at the end, i.e., if it is evicted or never inserted.

F. Why the presence bits help

A count-only policy faces a trade-off at every count: a larger $\pi(v)$ shortens the late insertion behind stale owners, $\sum_v 1/\pi(v)$, but raises q_v^{cnt} and with it the decay loss and the eviction probability of persistent owners. It can only shift this trade-off between count levels. The presence bits remove it.

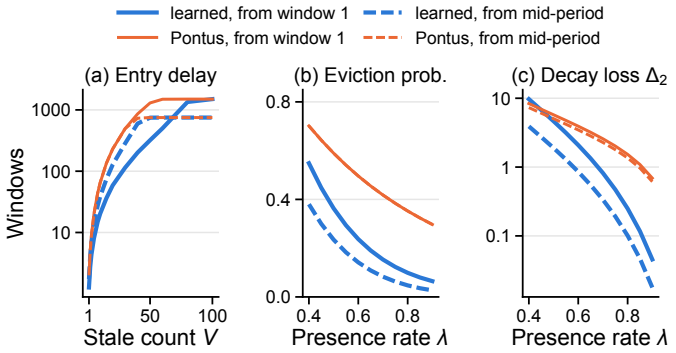


Fig. S1. Exact chains with the deployed table T and with the Pontus rule ($M = 1500$, one attempt per window, $\theta = 1/2$), for a flow that starts in window 1 (solid) or at mid-period (dashed). (a) Expected entry delay behind a stale owner of count V, up to the end of the period. (b, c) Eviction probability and expected decay loss of a surviving flow, versus its presence rate.

Corollary S1 (decoupling)**.** *Let π be a count-only policy that meets the conditions of Theorem S2, $\lambda_0 \in (0, 1)$, and π' the presence-aware policy with $\pi'_{00}(v) = \min\{1, \pi(v)/(1-\lambda_0)^2\}$ and $\pi'_{01} = \pi'_{10} = \pi'_{11} = 0$. For every flow with $\lambda \geq \lambda_0$, π' has a decay-loss bound* (S5) *and an eviction probability* (S6) *no larger than those of π, and behind a stale owner ($H = 00$) of count V, an expected entry delay of*

$$\frac{1}{\rho}\sum_{v=1}^{V} \max\left\{1, \frac{(1-\lambda_0)^2}{\pi(v)}\right\}.$$

Proof. By (S1), $q'_v = \rho(1-\lambda)^3\pi'_{00}(v) \leq \rho(1-\lambda)\pi(v)\,(1-\lambda)^2/(1-\lambda_0)^2 \leq q_v^{\mathrm{cnt}}$ for $\lambda \geq \lambda_0$, and likewise $\bar{\pi}'(1) \leq \pi(1)$; (S5) and (S6) are non-decreasing in every q_v and in $\bar{\pi}(1)$. The entry delay is (S3) with π'_{00}. □

At the persistence threshold $\lambda_0 = \alpha = 0.4$, $(1-\lambda_0)^2 = 0.36$: by knowing only the last two windows of an owner, a policy can release stale owners up to about 2.8× sooner (for the Pontus rule, 477 instead of 1,325 windows behind an owner of count 50 with $\rho = 1$) without raising the decay-loss bound or the eviction probability of any persistent flow. The learned policy is not of this extreme form, but it moves in the same direction: it decays owners with $H = 00$ 1.5–2.2× more often than the Pontus rule and owners with $H = 11$ 21–232× less often (Section S2).

G. Exact chains of the deployed table

The deployed table changes with the time bin, and the worst case over the bins is loose: the learned policy decays least late in the period, so its minimum of π_{00} is below the rule's. We therefore compute the exact, time-inhomogeneous Markov chains on the count and the presence bits with all 768 entries of T, including its count bins, without (A4); only (A1)–(A3) remain, with $\rho = 1$ and $\theta = 1/2$. Fig. S1 shows the results.

Entry. Behind a stale owner of count 10, a flow that starts in window 1 gets its bucket after 20 windows with the learned table and 65 with the Pontus rule; behind an owner of count 50, after 320 versus 1,298 windows, and under the rule it enters before the end of the period only with probability 0.80. From mid-period on, the learned table decays less, but it is still faster (39 versus 65 windows at count 10); behind an owner of count 50, neither policy is likely to free the bucket in the remaining 750 windows (probability 0.04 versus 0).

Eviction. A persistent flow inserted in window 1 is evicted with probability 0.55 versus 0.70 at $\lambda = 0.4$, 0.24 versus 0.50 at $\lambda = 0.6$, and 0.10 versus 0.35 at $\lambda = 0.8$; inserted at mid-period, 0.38 versus 0.70 at $\lambda = 0.4$. The advantage holds for both extremes of packet order: at $\lambda = 0.4$, the eviction probability is 0.45 versus 0.57 with $\theta = 0$ and 0.64 versus 0.82 with $\theta = 1$.

Decay loss. A flow that survives loses 2.1 versus 3.9 units at $\lambda = 0.6$ and 0.25 versus 1.5 at $\lambda = 0.8$. At $\lambda = 0.4$ from window 1, it loses more under the learned table (9.7 versus 8.4), because the learned table keeps marginal flows that the rule evicts; from mid-period on, it loses less (3.9 versus 7.3).

The learned table thus shortens the entry delay and lowers the eviction probability at the same time, as Corollary S1 predicts; the decay loss of surviving flows near the threshold is the price it pays early in the period.

H. Correctness of the FPGA shortcuts

The FPGA pipeline replaces the per-window sweep by a lazy roll-over and makes each decay draw one stage early (Section IV-A of the paper). A roll-over by one window maps (F, R, H) to $(0, 0, ((H \ll 1) \,|\, F)\,\&\,3)$.

Lemma S2 (lazy roll-over)**.** *(a) For $\delta \geq 1$, δ consecutive roll-overs map (F, R, H) to $\big(0, 0, ((H \ll \delta) \,|\, (F \ll (\delta - 1)))\,\&\,3\big)$. (b) With a 4-bit stamp, the pipeline rolls a bucket that was last accessed δ windows ago over by $\delta \bmod 16$ windows. Only if $\delta \geq 16$ and $\delta \bmod 16 \in \{0, 1\}$ can its state differ from that of the sweep.*

Proof. (a) By induction on δ: the case $\delta = 1$ is the definition, and a further roll-over clears F and shifts H, $((((H \ll \delta) \,|\, (F \ll (\delta - 1)))\,\&\,3) \ll 1)\,\&\,3 = ((H \ll (\delta + 1)) \,|\, (F \ll \delta))\,\&\,3$. (b) For $\delta \geq 2$, (a) gives $(0, 0, 00)$, since both bits of H are shifted out. If $\delta \geq 16$ and $\delta' = \delta \bmod 16 \geq 2$, the pipeline also rolls over by at least two windows and reaches the same state; only $\delta' = 0$ (no roll-over) and $\delta' = 1$ (one roll-over) can leave stale bits. □

In the bit-exact model, 17% of the reads that follow a gap of 16 or more windows change the state (Section S3); this exceeds $2/16$ because gaps of 16 and 17 windows are the most common long gaps.

Theorem S4 (speculative draw)**.** *Let the input of the pipeline idle for one cycle whenever the window changes, and let the controller write the thresholds only while the pipeline is empty. Then, for every packet, the pipeline makes the same decision and writes the same bucket words as Algorithm S1 with the lazy 4-bit roll-over, applied to the packets in order with the same hashes and random numbers. With the per-window sweep instead, the same holds except after the gaps of Lemma S2(b).*

Proof. (i) The LFSR advances once per valid input, so packet i draws the same random number as in the sequential order,

Algorithm S1 LEAP update for an arriving flow e in window t (Section III-A and Fig. 4 of the paper)

1: $g \leftarrow \mathrm{fp}(e)$; $m \leftarrow \perp$
2: **for** $i = 1$ to d **do**
3: $\quad B \leftarrow$ bucket $h_i(e)$ of row i
4: $\quad$ **if** B is empty **then** $B \leftarrow (g, P{=}1, F{=}1, R{=}0, H{=}00)$; **return**
5: $\quad$ **if** $B.\mathrm{fp} = g$ **then**
6: $\quad\quad$ **if** $B.F = 0$ **then** $B.P \leftarrow B.P + (B.R\ ?\ 2 : 1)$; $B.F \leftarrow 1$
7: $\quad\quad$ **return**
8: $\quad$ **if** $m = \perp$ **or** $B.P < m.P$ **then** $m \leftarrow B$
9: **if** $m.F{=}0 \wedge m.R{=}0 \wedge \mathrm{rnd}_{16} < T[\kappa(t)][m.H][b(m.P)]$ **then**
10: $\quad m.P \leftarrow m.P - 1$; $m.R \leftarrow 1$
11: $\quad$ **if** $m.P = 0$ **then** $m \leftarrow (g, P{=}1, F{=}1, R{=}0, H{=}00)$

and the thresholds do not change while it is in flight. (ii) A packet changes at most one of its buckets (the matching, empty, or minimum-count one) and only rolls the others over; so packet $i{-}1$ changed at most one bucket of packet i. (iii) In S4, packet i draws on the newest word of each of its buckets, which includes the writes of the packets two to four positions ahead, rolled over to the current window, and takes H and $b(P)$ from that word; the only write it can miss is that of packet $i{-}1$, which is in S5 in the same cycle. If packet $i{-}1$ did not change a bucket, the word S4 used equals the sequential state, because roll-overs compose (Lemma S2(a)). (iv) S5 re-evaluates the whole control flow of Algorithm S1 on the current words, including that of packet $i{-}1$: empty and matching buckets, row order, the minimum count with ties to row 1, and $F = R = 0$; only the comparison of the random number with the threshold comes from S4. If packet $i{-}1$ changed bucket b of packet i in the same window, it inserted, counted, decayed, or replaced the owner of b, and each of these sets F or R; in Algorithm S1, b then either matches or is not eligible for a decay, and S5 discards the draw for b. By (ii), the draw on the other bucket was made on its current state. (v) Packets $i{-}1$ and i in different windows are separated by the idle cycle, so packet $i{-}1$ is two positions ahead and its write reaches S4. □

S2. Training and the Learned Policy

Update and table. Algorithm S1 is the update of the paper; it differs from Pontus only in line 9, where the lookup in T replaces the probability $1/(P+1)$. At the end of each window, every bucket shifts F into H and clears F and R (lazily on the FPGA, Section S3). The table has 8 time bins, $\kappa(t) = \lfloor 8(t-1)/M \rfloor$, 4 values of H, and 24 count bins $b(P) = 2k + \beta$, where $k = \lfloor \log_2 P \rfloor$ and β is the bit below the leading one of P; each entry stores the policy at the bin's representative count and time as a 16-bit threshold.

Two stages. Stage 1 runs the Pontus rule on the January trace and logs 13.8 M decay opportunities; in a sample of 3 M of them, 5.4% concern an owner that ends persistent, and a logistic classifier predicts this label with 97.1% accuracy. The Stage-1 policy scales the Pontus probability by the predicted chance that the owner is *not* persistent. Stage 2 starts CMA-ES from this policy (population 8, step size 0.6) and scores each candidate by its AAE relative to the Pontus rule on the January trace at 16, 32, and 64 KB with two hash seeds. After 14 generations, a restart with the step size reset to 0.6 runs 12 more (about 40 minutes on four cores in total); the best candidate reaches 0.63 of the rule's AAE on the training trace. The policy never sees the test traces; the April trace was used in early design exploration, so February, March, May, July, and October are strictly held out.

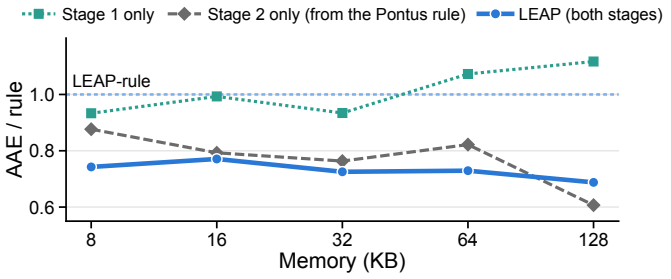


Fig. S2. AAE of each training stage relative to LEAP-rule (ratio of the mean AAEs over the four MAWI test traces, five seeds each). "Stage 2 only" runs a single 26-generation search from the Pontus rule instead of the Stage-1 policy.

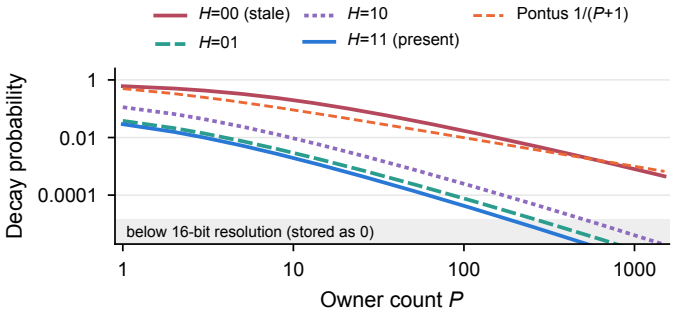


Fig. S3. Learned decay probability versus the owner's count for each presence pattern H (at mid-period, $\tau = 0.5$), compared with the Pontus rule.

What each stage contributes. Fig. S2 separates the two stages (Section V-C of the paper). Stage 1 alone barely helps and is worse than the rule at 64–128 KB: its hindsight labels judge only the owner, so it protects persistent owners even when they block other persistent flows. The search does most of the work: started from the Pontus rule instead, it still reaches most of the gain, but ends 5% worse than LEAP on average, 18% worse at 8 KB and 13% at 64 KB, while it is 12% better at 128 KB. Thus the Stage-1 start helps at 8–64 KB.

What the policy learned. Fig. S3 shows a "use it or lose it" rule. Owners absent in both previous windows ($H{=}00$) are decayed 1.5–2.2× more often than under the Pontus rule at counts 2–100, so stale owners release their buckets sooner. Owners present in both windows ($H{=}11$) are decayed 21×, 47×, and 232× less often at counts 2, 10, and 100, and recent presence ($H{=}01$) is worth more than older presence ($H{=}10$), although $H{=}10$ owners end persistent slightly more often (Fig. 2(b) of the paper): the search minimizes the AAE, not the error of a persistence label. Because protected owners are rarely decayed, LEAP also decays less often at 8–64 KB (on 1.8–7.6% of the packets versus 3.1–7.9% for LEAP-rule), which lowers the recirculation load on Tofino.

TABLE S1

STAGES OF THE FPGA PIPELINE (ONE PACKET PER CYCLE; THE ESTIMATE LEAVES 6 CYCLES AFTER THE PACKET ENTERS).

Stage	Work
S0	register the 64-bit key and the window id; advance the 16-bit LFSR
S1	H3 hashing: two row indices and the fingerprint (XOR trees)
S2	read both candidate words (32-bit bucket + 4-bit stamp)
S3	count bins $b(P)$ by leading-one detection; forwarding compares
S4	roll-over; speculative draw: LUT-RAM$[\{H, b(P)\}]$ vs. LFSR
S5	roll-over and decision with the word of the preceding packet
S6	write back both rows; output the flow's estimate

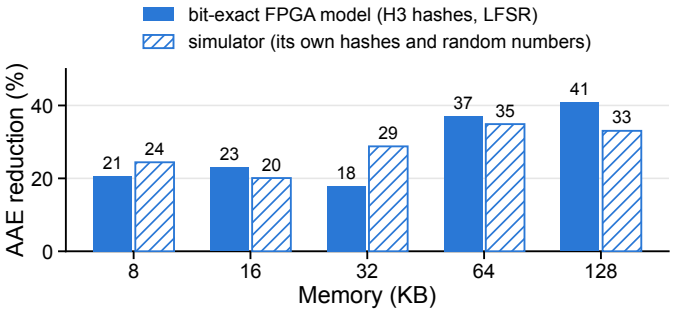


Fig. S4. AAE reduction of the learned table over the Pontus rule compiled into the same table, mean over the four MAWI test traces: bit-exact FPGA model (five draws of the H3 masks and the LFSR state) and the simulator (five seeds).

S3. FPGA PIPELINE

Table S1 lists the stages of Fig. 5 of the paper. Read-after-write hazards are resolved by forwarding: S4 uses the words written by the packets two to four positions ahead, and S5 the word of the immediately preceding packet; all forwarding selects are computed one stage early and registered, so no comparator lies on the critical path.

Lazy roll-over. A bucket that is not accessed for 16 or more windows sees its 4-bit stamp wrap, so it is rolled over by fewer windows than it should be. Running the bit-exact hardware model with and without this effect on the four test traces at 8–128 KB, 0.055% of the 1.2 G bucket reads follow such a gap (almost none below 64 KB, mostly at 128 KB, where many buckets stay idle), 17% of these reads change the state (Lemma S2), and the AAE changes by at most 0.7%.

Verification. The speculative draw is safe within a window because any update sets F or R, which forbids another decay of that bucket (Section IV-A of the paper). Across a window boundary, the roll-over clears both flags, so the input idles for one cycle (once per 20 K packets on MAWI); without this cycle, a stress trace with a window boundary every three packets and 16 buckets per row diverges from the reference model within the first few hundred packets, and with it every estimate matches. In Icarus Verilog, the RTL matches the bit-exact C++ model packet by packet on 400 K MAWI packets with the rule table, on 1 M packets with random tables and idle input cycles, and on the stress trace. The report variant of the core, which adds a threshold input and flags the packet whose own count reaches $\lceil \alpha M \rceil$, also matches the model on 300 K packets, including every report.

TABLE S2

PACKET-LEVEL MODEL OF THE COMPILED TOFINO PROGRAM VERSUS THE SIMULATOR AT 32 KB (AAE; MODEL: TEN SEEDS; SIMULATOR: FIVE SEEDS AND THEIR RANGE).

	April	July	October
Tofino model, immediate second pass	6.13	3.15	8.52
Simulator, mean	6.61	3.51	8.37
range over seeds	3.71–11.33	2.39–4.76	7.22–10.57
Packets that recirculate (model)	5.4%	6.9%	4.7%

The learned table on the hardware. The simulator, in which the policy is trained and evaluated, uses its own hash function and random numbers; the bit-exact model of the RTL uses the H3 hashes, the 16-bit LFSR, and the 4-bit stamps of the FPGA. Fig. S4 compares what the learned table gains over the compiled rule in both: 18–41% on the hardware model and 20–35% in the simulator, with no consistent difference across memory sizes. The absolute AAE of the hardware model is 8–12% higher than the simulator's on average over the traces, while single draws vary as widely as single simulator seeds.

Resources. Table II of the paper gives the ECP5 cost from 8 to 128 KB. At 32 KB, the threshold LUT-RAM uses 64 distributed-RAM cells (16 for the rule), and Yosys synthesis for an UltraScale+ device needs 1,159 LUTs and 646 flip-flops (1,014 and 642 with the rule).

S4. TOFINO PROGRAM

The P4 program runs Algorithm S1 in the ingress pipeline of Tofino 1 (Fig. 6 of the paper). Stage 0 hashes the key (CRC32 for the indices, CRC32C for the fingerprint) and draws the random number, and stage 1 reads the window that the controller advances. Row 1 occupies stages 1–4; a table in stage 2 decides whether row 2, in stages 3–6, is needed; stages 5–8 pick the minimum-count owner, stage 9 compares its threshold in T with the random number, and stage 10 issues the decay request.

Window state. Each row keeps five register arrays: the fingerprint, the count, the last window in which the owner was seen ($last$), whether it was also seen in the window before ($prev$), and the window of its last decay (dec). With $\delta = t - last$, the flags of the paper are $F = (\delta = 0)$, $H_1 = (\delta = 1)$, $H_2 = (\delta = 2) \vee (\delta = 1 \wedge prev)$, and $R = (dec = t)$; a 4-entry ternary table computes F and H. The stamps are 16 bits wide: with 8-bit stamps, a decay stamp aliases to the current window every 256 windows and wrongly turns the owner's next arrival into a $+2$ increment.

Second pass. A decay is decided only in stage 9, after the packet has passed both rows, so the packet recirculates a request that carries the row, the index, the owner's fingerprint, and the window. On its second pass, the request decays the owner only if the fingerprint is unchanged and the owner has neither arrived nor been decayed in that window, and it replaces the owner if its count was 1; a request whose bucket changed in the meantime is discarded. In a deployment, the request travels as a truncated mirrored copy, and the original packet is forwarded unchanged.

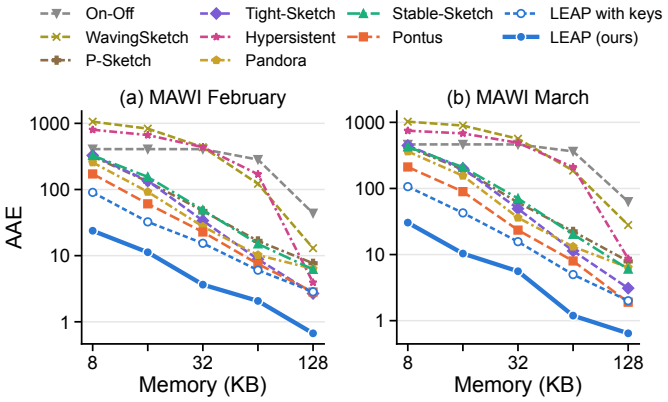


Fig. S5. AAE on the two held-out MAWI traces of February and March 2026 (mean of five seeds).

Does the switch behave like the simulator? Our packet-level model executes the compiled program's register actions in pipeline order and delivers each decay request after a configurable number of packets. With the packed flags in place of the window stamps, its decisions are identical, so the encoding is exact. Table S2 shows that its AAE lies within the simulator's range over hash seeds on all three traces; delaying the second pass by 128 or 1,024 packets changes the AAE by less than 1%, while 3.7% and 8.7% of the requests arrive stale and are discarded. On Intel's Tofino model, a reduced build (256 buckets per row, random numbers taken from the packet so that the run is deterministic) loaded under `bf_switchd` reproduces all 2,560 register words of our C++ reference after 6,000 and 40,000 MAWI packets; after 40,000 packets, its AAE of 1.98 lies within the range of the simulator over 30 seeds of the same configuration (1.06–2.26).

Compilation. bf-p4c 9.13.4 places the program in 11 of the 12 stages (Table I of the paper); a first version with `if` statements around the register actions needed 17 stages, because each branch made the rest of the pipeline wait. The build that reports persistent flows as digests fits into the same 11 stages.

S5. Additional Results

More backbone traces. On February and March 2026 (Fig. S5), LEAP is 3.0–8.6× more accurate than the best baseline at every memory size, the same picture as on the four test traces of the paper.

Fingerprint bits for presence. Taking the two presence bits from the fingerprint does not cost accuracy: against a version with the same number of buckets whose fingerprint keeps the two bits, the AAE changes by -1.2% on average over the four test traces, without a consistent sign across memory sizes.

Hand-crafted rules. The learned policy suggests a simple shape: decay stale owners more and present owners less. Table S3 lists five rules with this shape; rule E has the lowest AAE relative to the Pontus rule on the January trace and is the one compared in Fig. 8(b) of the paper. The constants matter: the two rules that always decay stale owners (A, D) are 1.4–2.5× worse than the Pontus rule. On the test traces, LEAP is better than rule E by 18% at 8 KB (95% CI 14–23%) and by 8% at 16 KB (1–15%), paired by trace and hash seed; at 32–128 KB the difference is not significant (rule E is 11% better at 128 KB, $p = 0.08$).

TABLE S3
Hand-crafted presence rules on the January training trace (AAE, three seeds). Each rule multiplies the Pontus probability by a factor for owners absent in both previous windows, present in one, and present in both; "always" decays with probability 1.

Rule	16 KB	32 KB	64 KB
Pontus rule $1/(P+1)$	21.94	7.05	3.28
A: always, 1, 0	53.10	16.55	4.52
B: 4, 1, 1/4	16.48	5.56	1.80
C: 2, 1, 0	17.70	5.89	2.09
D: always, 1, 1/4	54.17	16.50	4.50
E: 4, 1, 0 (chosen)	17.33	**5.08**	1.67
LEAP (learned)	**15.34**	5.76	**1.55**

Reporting persistent flows. LEAP stores fingerprints, so it cannot list persistent flows from its buckets; instead, the pipeline reports a flow when one of its packets lifts its count to $\lceil \alpha M \rceil$, and that packet carries the key. On the MAWI test traces and CTU-13, no report was a false positive, and a flow was reported twice only when it was decayed below the threshold and crossed it again, which was rare. Reporting is also more precise than querying the sketch at the end of the stream: both find the same persistent flows, but a non-persistent flow whose fingerprint matches a persistent owner is never reported, which raises the F1 score at 8 KB from 0.913 to 0.932 on MAWI. The controller never receives more reports than there are persistent flows (at most 944 per trace).

Baseline settings. P-Sketch and Tight-Sketch use 3 rows and Pandora 4, as in their implementations; WavingSketch spends half of its memory on a Bloom filter that deduplicates arrivals per window. Hypersistent Sketch was designed for windows of 1,500 packets; with 20 K packets per window, its filter counters saturate, so we tuned its memory split on the January trace and use an 80% filter share with threshold 255 (its artifact: 60%).

S6. Artifact

The anonymous repository [1] contains LEAP on the three platforms: a C++ implementation for CPUs, the FPGA RTL with its bit-exact model and test benches, and the P4 program for Tofino with its control-plane script, together with the learned table and the code that trains it.

References

[1] Anonymous, "LEAP: Technical report and code," https://anonymous.4open.science/r/Leap-DAC/, 2026.